

Material de Aula: Fundamentos da Linguagem Java

Dani Juliano Czelusniak, Prof. Dr.

Atualizado em 2026

1 Análise Teórica e Contextualização Pedagógica

O aprendizado de uma nova linguagem de programação exige a compreensão de seus blocos fundamentais. No ecossistema Java, um dos pilares do paradigma de Programação Orientada a Objetos (POO), a estruturação rígida de tipos e a organização do código são essenciais para a construção de softwares robustos. Este material de estudo foi elaborado para conduzir o discente através dos conceitos essenciais da linguagem Java, seguindo um fluxo de aprendizado que abrange desde o armazenamento em memória com variáveis, passando pela interação com o usuário via terminal, uso de bibliotecas matemáticas nativas, até o encapsulamento de tudo isso na estrutura obrigatória de uma aplicação mínima.

2 Variáveis e Tipos Numéricos

Diferentemente de linguagens com tipagem dinâmica, Java exige que toda variável tenha seu tipo explicitamente declarado antes de ser utilizada. Isso garante segurança de memória e otimização em tempo de compilação.

2.1 Tipos Inteiros e Reais

A linguagem Java fornece diferentes tipos numéricos para otimizar o uso da memória RAM. Para valores inteiros, temos os seguintes tipos básicos de dados:

Tabela 1: Tipos Primitivos Inteiros em Java

Tipo	Bits	Valor Mínimo	Valor Máximo
<code>byte</code>	8	-128	127
<code>short</code>	16	-32768	32767
<code>int</code>	32	-2147483648	2147483647
<code>long</code>	64	-9223372036854775808	9223372036854775807

Para valores numéricos com casas decimais (reais), a linguagem dispõe do tipo `float` (32 bits), que requer a letra 'F' ou 'f' como sufixo (ex: `3.14f`), e o tipo `double` (64 bits), que oferece o dobro de precisão.

Além disso, variáveis podem ser promovidas (conversão implícita) de um tipo menor para um maior, ou rebaixadas (conversão explícita, via *cast*) de um tipo maior para menor, com possível perda de dados. Variáveis também podem ser transformadas em constantes imutáveis através da palavra reservada `final`.

```

1 public static void main(String[] args) {
2     int k = 10;           // Variavel do tipo inteiro
3     long z = 2081;        // Variavel do tipo inteiro longo
4
5     // Operadores aritmeticos: + - * / %
6     int resultado = k * 7 - 5;
7
8     // Conversao Implicita
9     short s = 10;
10    int i = s;
11
12    // Conversao Explicita (Cast)
13    double d = 8.6;
14    int m = (int) d;       // m recebe 8 (trunca a parte decimal)
15
16    // Constantes
17    final float PI = 3.14f;
18 }

```

Listing 1: Declaração de variáveis, operadores e conversão

Explicação Linha a Linha:

1. **Linhas 2 e 3:** Declaração e atribuição simultânea de variáveis inteiras.
2. **Linha 6:** Expressões matemáticas utilizam a precedência padrão. Os operadores suportados incluem adição, subtração, multiplicação, divisão e módulo (%).
3. **Linhas 9 e 10:** Demonstração de conversão implícita. Um `short` cabe perfeitamente dentro de um `int`, logo, não é necessária sintaxe extra.
4. **Linhas 13 e 14:** Conversão explícita. Como `double` ocupa 64 bits e `int` apenas 32, o programador deve assumir a responsabilidade pela possível perda de dados declarando `(int)` antes da variável.
5. **Linha 17:** A palavra `final` garante que `PI` será constante durante toda a execução.

3 Entrada e Saída de Dados

A interatividade de aplicações no terminal se dá por meio de fluxos de entrada e saída. Em Java, a leitura de dados fornecidos pelo usuário via dispositivo padrão (o teclado) é viabilizada pela classe `Scanner`, presente na biblioteca `java.util`.

```

1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner teclado = new Scanner(System.in);
6
7         System.out.print("Digite a sua idade: ");
8         int k = teclado.nextInt();

```

```

9
10     System.out.println("Voce tem " + k + " anos");
11 }
12 }

```

Listing 2: Leitura e Escrita de Dados Interativa

Explicação Linha a Linha:

1. **Linha 1 (`import java.util.Scanner`):** Diretiva obrigatória no topo do arquivo que importa a classe de leitura de dados que não está no escopo básico da linguagem.
2. **Linha 5 (`Scanner teclado = new Scanner(System.in)`):** Instanciação do objeto `Scanner`. O parâmetro `System.in` orienta o objeto a escutar o *stream* de entrada padrão do sistema operacional.
3. **Linha 7 (`System.out.print`):** Imprime um texto no terminal sem quebrar a linha ao final.
4. **Linha 8 (`teclado.nextInt()`):** Bloqueia a execução do programa até que o usuário digite um valor inteiro e pressione ENTER, convertendo a entrada em `int` e salvando na variável `k`. Há também `nextFloat()`, `nextDouble()`, etc.
5. **Linha 10 (`System.out.println`):** Concatena a *string* com o valor da variável `k` e imprime na tela quebrando a linha ao final (`\n`).

4 Biblioteca Math

Para operações matemáticas avançadas que extrapolam a aritmética básica, o Java fornece a classe `Math`, que não necessita de importação prévia. Ela oferece dezenas de métodos estáticos e constantes indispensáveis em rotinas numéricas.

```

1 public static void main(String[] args) {
2     double base = 3.0;
3     double expoente = 2.0;
4
5     // Potenciacao e Radiciacao
6     System.out.println(Math.pow(base, expoente)); // 9.0
7     System.out.println(Math.sqrt(64));           // 8.0
8
9     // Constantes e Modulos
10    System.out.println(Math.PI);
11    System.out.println(Math.abs(-6));             // 6
12
13    // Sorteio
14    System.out.println(Math.random());            // 0.0 a 0.999...
15 }

```

Listing 3: Utilização de métodos matemáticos

Explicação Linha a Linha:

1. **Linha 6 (`Math.pow`):** Efetua a operação de base elevada ao expoente. Sempre retorna um tipo `double`.
2. **Linha 7 (`Math.sqrt`):** Calcula a raiz quadrada do valor fornecido.
3. **Linha 10 (`Math.PI`):** Acessa a constante que representa o valor de Pi com alta precisão (3.14159...). Da mesma forma, existe `Math.E`.
4. **Linha 11 (`Math.abs`):** Retorna o valor absoluto de um número (módulo), convertendo negativos para positivos.
5. **Linha 14 (`Math.random`):** Gera um valor de ponto flutuante pseudoaleatório entre 0.0 (inclusivo) e 1.0 (exclusivo).

5 Aplicação Mínima na Linguagem Java

Todo código em Java é estritamente organizado em classes. Diferentemente de linguagens puramente procedurais, não há código "solto" em Java. Toda aplicação Java consiste em um conjunto de classes, e a execução do programa sempre se inicia através de um método específico contido em alguma delas, chamado `main`.

A formatação de um código Java é livre, o que significa que o espaçamento (vertical ou horizontal) não possui significado semântico (ao contrário do Python, onde a indentação demarca escopos). Escopos em Java são delimitados por chaves `{}` e comandos são finalizados com ponto-e-vírgula `;`.

```
1  /*
2      Classe de teste - Todo arquivo fonte (.java)
3      e compilado para bytecode (.class) e executado pela JVM.
4  */
5  class Teste {
6      // Ponto de entrada do programa
7      public static void main(String[] args) {
8          System.out.println("Objetos");
9      }
10 }
```

Listing 4: Estrutura base e comentários de uma aplicação Java

Explicação Linha a Linha:

1. **Linhas 1 a 4 (`/* ... */`):** Bloco de comentários de múltiplas linhas. Comentários não são compilados para o bytecode `.class`.
2. **Linha 5 (`class Teste`):** Definição de uma classe, cujo nome (identificador) é `Teste`. Por convenção (e regra para classes públicas), o nome do arquivo `.java` deve coincidir com o nome da classe principal.
3. **Linha 6 (`// ...`):** Comentário de linha única.

4. **Linha 7 (`public static void main(String[] args)`):** Esta é a assinatura clássica do método principal. Ele é assemelhado a uma função, mas pertence à classe. O qualificador `public` garante visibilidade externa, `static` permite chamá-lo sem instanciar a classe, `void` significa que ele não retorna dados ao sistema, e `String[] args` é o parâmetro para recepção de argumentos via linha de comando.
5. **Linha 8 (`System.out.println(...)`):** Comando básico de impressão no dispositivo de saída padrão. Todo comando simples na linguagem é encerrado obrigatoriamente por `;`.

6 Estruturas de Controle de Fluxo

O paradigma de programação estruturada baseia-se em três construções fundamentais: sequência, decisão e iteração. Até o momento, os códigos apresentados seguiram um fluxo puramente sequencial. A seguir, exploraremos como desviar esse fluxo e criar laços de repetição utilizando apenas estruturas procedurais dentro do método principal, garantindo um código limpo e direto.

6.1 Estruturas de Decisão: `if`, `else if` e `else`

A estrutura `if` avalia uma expressão booleana. Se a condição for verdadeira (`true`), o bloco de código delimitado pelas chaves é executado. A cláusula `else if` permite encadear múltiplas condições de forma mutuamente exclusiva, enquanto o `else` atua como um bloco de execução padrão, acionado caso todas as condições anteriores sejam falsas (`false`).

```
1 public static void main(String[] args) {  
2     double nota = 7.5;  
3  
4     // Avalia as condicoes de cima para baixo  
5     if (nota >= 7.0) {  
6         System.out.println("Aprovado");  
7     } else if (nota >= 4.0) {  
8         System.out.println("Exame Final");  
9     } else {  
10        System.out.println("Reprovado");  
11    }  
12 }
```

Listing 5: Estrutura de decisão para faixas de valores

6.2 Estruturas de Seleção: `switch/case`

Quando é necessário comparar o valor exato de uma mesma variável com múltiplos valores discretos (como inteiros, caracteres ou *strings*), o comando `switch` oferece uma sintaxe mais limpa e performática que o encadeamento de múltiplos `if/else`. O comando `break` é fundamental para interromper a execução do bloco e evitar que os casos subsequentes sejam executados em cascata (efeito conhecido como *fall-through*).

```

1 public static void main(String[] args) {
2     int opcaoMenu = 2;
3
4     switch (opcaoMenu) {
5         case 1:
6             System.out.println("Acessando Cadastro...");
7             break;
8         case 2:
9             System.out.println("Gerando Relatorio...");
10            break;
11        default:
12            System.out.println("Opcao invalida.");
13            break;
14    }
15 }

```

Listing 6: Estrutura de seleção para menus

6.3 Estruturas de Repetição (Laços)

Os laços de repetição (ou malhas) permitem executar um bloco de código iterativamente enquanto uma condição de controle permanecer válida, o que é essencial para processamento de lotes de dados ou validação contínua.

6.3.1 Laço while

O comando **while** é caracterizado por realizar o teste condicional **antes** de cada iteração (pré-testado). Se a condição inicial for falsa logo na primeira verificação, o bloco interno pode nunca ser executado. É ideal para situações onde não se sabe previamente o número exato de repetições.

```

1 public static void main(String[] args) {
2     int contador = 1;
3
4     // A condicao e checada antes de entrar no bloco
5     while (contador <= 3) {
6         System.out.println("Iteracao: " + contador);
7         contador++; // Incremento para evitar loop infinito
8     }
9 }

```

Listing 7: Laço de repetição pre-testado (while)

6.3.2 Laço do/while

Diferentemente do **while**, o laço **do/while** realiza o teste condicional **após** a execução do bloco (pós-testado). Isso garante que as instruções internas sejam executadas, no mínimo, uma única vez, independentemente de a condição ser verdadeira ou falsa. É a estrutura mais indicada para entrada e validação de dados de formulários ou menus interativos.

```

1 import java.util.Scanner;
2
3 public class Validacao {
4     public static void main(String[] args) {
5         Scanner entrada = new Scanner(System.in);
6         int senha;
7
8         do {
9             System.out.print("Digite a senha numerica (1234): ");
10            senha = entrada.nextInt();
11        } while (senha != 1234); // Repete enquanto a senha for
12            incorreta
13
14        System.out.println("Acesso liberado.");
15    }
16 }

```

Listing 8: Laço de repetição pos-testado (do/while)

6.3.3 Laço for

O comando `for` agrupa a inicialização da variável de controle, a condição de parada e o passo de incremento/decremento em uma única linha. Ele é a estrutura ideal para laços contados, ou seja, quando se sabe de antemão exatamente quantas iterações devem ocorrer.

```

1 public static void main(String[] args) {
2     // Para i iniciando em 1, ate 5, de 1 em 1
3     for (int i = 1; i <= 5; i++) {
4         System.out.println("Valor do indice: " + i);
5     }
6 }

```

Listing 9: Laço de repetição contado (for)

6.3.4 Laço foreach (*Enhanced for loop*)

O `foreach` (introduzido no Java 5) é uma versão simplificada do `for` dedicada exclusivamente à iteração sobre coleções de dados ou arranjos (*arrays*). Ele abstrai e oculta o controle numérico do índice, extraindo diretamente os valores em sequência estruturada, o que previne erros de violação de memória ou estouro de limite (*IndexOutOfBoundsException*).

```

1 public static void main(String[] args) {
2     // Declaracao de um array estatico de inteiros
3     int[] valores = {10, 20, 30, 40, 50};
4
5     // Le-se: "Para cada inteiro 'v' dentro do array 'valores'"
6     for (int v : valores) {
7         System.out.println("Valor extraido: " + v);
8     }
9 }

```

7 Exercícios Resolvidos

Para fixar os conteúdos propostos de variáveis, I/O, funções matemáticas e a estrutura básica de classes, analise a implementação dos três problemas integrados abaixo.

7.1 Exercício 1: Cálculo de Área de Círculo

Enunciado: Construa uma aplicação em Java que solicite ao usuário a digitação do raio de um círculo (valor real) e, utilizando a classe `Math`, imprima a área calculada.

```
1 import java.util.Scanner;
2
3 class AreaCirculo {
4     public static void main(String[] args) {
5         Scanner leitor = new Scanner(System.in);
6
7         System.out.print("Informe o raio do círculo: ");
8         double raio = leitor.nextDouble();
9
10        double area = Math.PI * Math.pow(raio, 2);
11        System.out.println("A área do círculo é: " + area);
12    }
13 }
```

Listing 11: Resolução - Área de Círculo

7.2 Exercício 2: Teorema de Pitágoras

Enunciado: Leia os valores dos catetos A e B de um triângulo retângulo e calcule o valor da hipotenusa, mostrando o resultado truncado como inteiro.

```
1 import java.util.Scanner;
2
3 class Hipotenusa {
4     public static void main(String[] args) {
5         Scanner tc = new Scanner(System.in);
6
7         System.out.print("Cateto A: ");
8         double catA = tc.nextDouble();
9         System.out.print("Cateto B: ");
10        double catB = tc.nextDouble();
11
12        // h^2 = a^2 + b^2 -> h = sqrt(a^2 + b^2)
13        double hipot = Math.sqrt(Math.pow(catA, 2) + Math.pow(catB,
14        2));
15
16        int hipotInteira = (int) hipot; // Conversão explícita
```

```

16     System.out.println("Hipotenusa truncada: " + hipotInteira);
17 }
18 }

```

Listing 12: Resolução - Hipotenusa com Cast

7.3 Exercício 3: Cálculo Dinâmico de Idade

Enunciado: Declare uma constante para o ano atual e uma variável para receber o ano de nascimento do usuário através do console. Calcule e imprima a idade e o limite de batimento cardíaco da pessoa (cálculo: absoluto de 220 menos a idade).

```

1 import java.util.Scanner;
2
3 class CalculoIdade {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6         final int ANO_ATUAL = 2026;
7
8         System.out.print("Ano de nascimento: ");
9         int anoNascimento = sc.nextInt();
10
11         int idade = ANO_ATUAL - anoNascimento;
12         int frequenciaMaxima = Math.abs(220 - idade);
13
14         System.out.println("Sua idade e: " + idade);
15         System.out.println("BPM maximo: " + frequenciaMaxima);
16     }
17 }

```

Listing 13: Resolução - Idade e Frequência Cardíaca